Making your major project stronger: additional ideas and how to choose them

14 October 2026

1 Executive summary

The best "extras" for a major project are usually not new features. They are the non-technical qualities that assessors already mark and that students often do poorly: evaluation, testing, documentation, ethics and planning. Three accreditation bodies in three countries, and several universities' marking rubrics, name these qualities [1]–[8]. Studies of student projects find that students struggle most with exactly this kind of work, such as time management, documentation and requirements [9]–[12].

This report suggests 11 additional ideas, most linked to a standard or rubric that values them. The main points are:

- **Pick additions that improve something assessors mark,** and back each one with evidence such as test results or user findings. Extras without evidence earn little credit [4], [5], [11], [13].
- **Start with the cheap version of each idea.** A basic accessibility check or a five-person user test is likely to cost far less than full conformance or a statistical study [13], [14].
- **Protect the core project first.** Use a simple priority method (MoSCoW, explained in section 4), keep a time buffer, and agree your scope with your supervisor early [6], [7], [12], [15].

No source measured how much any addition raises a grade. The advice rests on what standards and rubrics reward, not on outcome data.

2 What makes a major project strong

Assessors reward more than a working product. Five qualities appear again and again (confidence: high):

- **Evaluation and analysis of results.** BCS, the UK professional body for information technology (IT), asks for "evaluation (with hindsight) of the project outcome and the process of its production" and "critical self-evaluation" [2]. Trinity College Dublin gives 15% of the mark to testing, evaluation and conclusions [4]. Oxford's top band asks for "a significant amount of analysis or assessment of results" [5].
- **Testing, or verification and validation.** BCS asks the report to explain "how verification and validation were applied" [2]. ABET, the US engineering accreditor, expects students to "develop and conduct appropriate experimentation, analyze and interpret data" [1].
- **Documentation and communication.** Engineers Australia expects "high quality engineering documents such as progress and project reports" [3].

- **Ethics and professional issues.** ABET, BCS and Engineers Australia all expect them [1]–[3].
- **Planning and project management.** Warwick gives 10% of the mark to a separate project-management report [6]. University College London (UCL) lists "realistic work commitments" as a learning outcome [7].

The core artefact still matters most. At Trinity, technical content and execution carry 50% of the mark [4]. Top bands also reward originality [4], [5].

Students often fall short on the non-technical work (confidence: medium). A study of 79 post-mortem reports (end-of-project reviews) from software engineering capstone teams at one university found that "technical issues tend to be less challenging than non-technical ones". Timeliness was the most common organisational issue (26.15% of codes in that category), and documentation (21.77%) and requirements (20.16%) were the most common professional and technical issues [9]. A 2010 survey of 88 engineering seniors found scheduling conflicts and lack of time management were the top barriers [10]. Other reports point the same way: students start late [11], struggle with time management [12] and present conclusions with "barely any evidence" [11], and introductory students do not test systematically while believing they do [16]. Most of these studies cover one institution, so treat them as signals, not proof.

3 Additional ideas

Table 1 lists 11 ideas. "Effort" is marked **(J)** where the rating is the authors' judgement; only four ideas have any source for their cost. Several ideas name a standard to design and test against. ABET asks for "appropriate engineering standards" in the final design experience [1], so citing the Web Content Accessibility Guidelines (WCAG), the Open Worldwide Application Security Project (OWASP) or Software Carbon Intensity (SCI) explicitly is a small extra that strengthens ideas 3, 5 and 9.

Table 1. Additional ideas for a major project

Idea	Why it helps	Effort (low / medium / high)	Sources
1. Stronger evaluation against your objectives, plus critical reflection on your process	Required by BCS; 15% of the mark at Trinity; reflective writing assessed at Warwick. Arguably core work: do this well first	Medium (J)	[2], [4]–[6], [17]
2. Small rounds of user testing (about five users per round, thinking aloud)	Gives evidence for evaluation and validation. Five users find most problems in a *qualitative* test	Low–medium; medium if ethics approval is needed	[1], [2], [14], [18], [19]
3. Accessibility pass aiming at WCAG 2.2 level AA: fix contrast, image descriptions (alt text), form labels, empty links and buttons, page language; report automated and manual checks	WCAG is the international web standard; failures are very common on real sites	Low (basic pass); high (claiming full conformance)	[13], [20]

Idea	Why it helps	Effort (low / medium / high)	Sources
4. Automated unit tests that run in continuous integration (CI) on every code push	BCS asks for verification and use of tools; students tend to under-test	Medium	[2], [16], [21], [22]
5. Security review against the OWASP Top 10, recording each risk and how you handle it	A widely used industry list of web security risks	Medium (J)	[23]
6. Ethics, privacy and professional-issues section	Expected by three accreditation bodies	Low (written section); medium if a user study needs approval (J)	[1]–[3], [19], [24]
7. Risk register: list risks, rate them, plan responses, review regularly	Engineers Australia expects risks to be identified and managed; late failures are hard to recover	Low (J)	[3], [25]
8. Project-management record and documentation (plan, progress log, README or user guide)	Assessed directly at Warwick (10%); documentation is a top student weakness	Low–medium (J)	[3], [6], [9]
9. Sustainability analysis: a written discussion, or an SCI estimate	ABET and Engineers Australia name environmental factors; SCI is a method standardised by the International Organization for Standardization (ISO)	Low (discussion); medium–high (measured SCI) (J)	[1], [3], [26]
10. Deployment to real users or a live environment	Indirect support only: BCS values meeting "a real need in a wider context"	Medium–high (J)	[2], [8]
11. Budget or cost estimate (engineering projects)	Graduates felt least prepared for budgeting (mean 5.0 on a 7-point scale; small, preliminary survey)	Low–medium (J)	[27]

Source: this report's analysis of the cited sources. (J) = authors' judgement; no source gives a cost. Ratings without (J) draw partly on a cost source, but the low / medium / high labels are still our reading.

Some terms in the table, and what the evidence says about cost:

- **User testing.** Five users are enough only for qualitative testing, where you watch people and note problems. Claims with numbers usually need more than 30 users [14]. The older five-user rule comes from a 2000 article and early-1990s assumptions [14], [18]. Testing with people may also need ethics approval. At one university department, referred cases "can take up to three weeks (longer if revisions are required)" [19].
- **WCAG** 2.2 is the web accessibility standard published by the World Wide Web Consortium (W3C). It has three conformance levels: A, AA and AAA [20]. In 2026, 95.9% of the top one million home pages had detected failures [13]. Six error types made up 96% of detected errors, so fixing them is a cheap first pass [13]. But "absence of detected errors does not indicate that a page is accessible or conformant" [13].

- **CI** (continuous integration) means tests run automatically each time you save code to a shared repository. In one small study, most students found writing their own tests helpful [21]. Testing has a cost, though. Professional teams using test-driven development (writing tests before the code) took 15–35% longer, but had 60–90% lower defect density (bugs per amount of code) [22]. That data is from 2009 and from professionals, so treat it as a rough pointer only. Focus tests on complex code rather than chasing coverage figures [22].
- **OWASP** publishes the Top 10, "a standard awareness document" for web security [23]. We used the 2021 edition; a 2025 draft was not checked.
- **SCI** is calculated as (E × I + M) per R: energy used, times the carbon intensity of that energy, plus emissions from making the hardware, per unit such as one user or one request [26].
- **Ethics timing.** Sussex expects students to have "considered ethical issues relating to their project" by the interim report [24]. Educators list ethics approval as a common project-module challenge [28].

4 How to choose and scope your additions

MoSCoW is a priority method from the Agile Business Consortium. It sorts work into **M**ust have, **S**hould have, **C**ould have and **W**on't have this time [15]. Must Haves should take no more than about 60% of your effort, and about 20% should sit in Could Haves as a buffer you can drop [15]. This guidance comes from industry and has not been tested on student projects.

How ambitious should you be? Supervisors disagree (confidence: low-medium):

- A Cambridge supervisor advises proposing "a relatively modest project" and listing extensions separately [17].
- One supervisor, writing on his personal blog, says most students are "insufficiently ambitious" and should go "deeper rather than wider" instead of bolting on features [29].
- The University of Sussex tells supervisors to encourage students "to be as ambitious as possible" while planning for setbacks [24].

Rubrics reward both originality [4], [5] and "realistic work commitments" [7]. A reasonable reading is this: be ambitious and deep in the core problem, promise only a modest set of Must Haves, and treat extras as Could Haves. This reconciliation is our interpretation.

A step-by-step prioritisation method. Steps 2–4 are our own synthesis of the sources, not a tested method.

1. **Secure the core first.** Write down your project aims. These are your Must Haves; keep them to about 60% of your estimated effort and keep them modest [15], [17].
2. **List possible additions** from Table 1. Note which marked quality each one improves: evaluation, testing, ethics, documentation or use of standards [2], [4], [5].
3. **Rate each addition** for value (does it link to the marking criteria?) and effort. Choose the cheap version first: a basic accessibility pass before full conformance, qualitative testing before statistics [13], [14].
4. **Sort them.** High value and low effort become Should Haves. The rest become Could Haves, the first to drop. Record what you are leaving out as Won't Have [15].

5. **Keep a time buffer and start slow items early.** Estimates tend to run short. In one 1994 study, students expected their thesis to take 33.9 days; it took 55.5 [30] (low-medium confidence: reported second-hand). Apply for ethics approval early [19], [28].
6. **Agree your scope in writing with your supervisor** and check progress at fixed points, such as a project plan, an interim report and a mid-project demonstration [6]–[8], [12]. If you fall behind, drop Could Haves first.
7. **Back up every addition you keep** with evidence: test results, an audit record or user findings. Cut anything you cannot evidence, or report it honestly as incomplete [4], [5], [11], [13], [17].

5 Risks and how to manage them

Adding extras has real risks (confidence: medium).

Table 2. Risks of adding extras

Risk	How to manage it
Running out of time. Project work "can consume far more time than its assessment-worth" [25]; time management is the most consistent student weakness [9]–[12]	Keep a Could-Have buffer and set early milestones [6]–[8], [15]
Late failure. Failures "discovered near the end of a large project are far more difficult to retrieve" [25]	Use interim reports and demonstrations [7], [8]; share unfinished work early [17]
Extras done for show. Students have admitted "making up content" for required process documents [11]; automated checks alone do not prove accessibility [13]	Evaluate each addition, or drop it
Extras crowding out the core, which carries the largest share of marks (50% at Trinity) [4]	Use MoSCoW order; go deep, not wide [29]. This is an inference from one rubric
Ethics approval delays [19], [28]	Apply early
Changing scope midway	Not harmful in itself. In one capstone course, about 46% of projects changed substantially, with no statistical difference in outcomes [31]. How you manage change matters more (confidence: low-medium)

Source: this report's analysis of the cited sources.

6 How this report was made

This report came from a structured debate between two researchers, who worked separately on the same brief:

- The **primary researcher** ("John") focused on accreditation standards, university marking rubrics and published standards.
- The **independent researcher** ("Jane") deliberately took a different angle: education studies, supervisors' advice, and evidence on cost and effort.

Each then rebutted the other's report. An analyst judged both on five criteria, weighting evidence highest.

Result: the primary report won, 4.00 to 3.55. It scored 4 out of 5 on every criterion. The independent report scored 4 on argument, content and formatting, and 3 on points covered and references. The winner answered all four research questions from the brief's preferred sources. The independent report left out assessors' own documents, had gaps on sustainability and deployment, and relied more on weak or older sources.

This report presents the winning research, plus nine points carried over from the independent report:

1. The MoSCoW guidance is dated 2026, not undated [15].
2. Ethics review can take up to three weeks, longer with revisions; Sussex expects ethics to be considered early [19], [24].
3. Five test users suit only qualitative tests [14].
4. Automated testing costs time [22].
5. Students fall short mainly on non-technical work [9], [10].
6. Supervisors disagree on how ambitious to be [17], [24], [29].
7. Six error types cover most detected accessibility errors [13].
8. Students and new graduates underestimate time and difficulty (low-medium confidence) [30], [32].
9. Avoid the claim that most software features go unused; it rests on four internal applications [33].

7 Limitations

- **No outcome data.** No source shows how much any addition raises a grade.
- **Effort ratings are mostly judgement.** Only four ideas have a cost source, and each is a single source. The testing figures are from 2009 professional teams [22]; the ethics timing is from one non-computing department [19].
- **Limited geography.** Rubrics come from the UK and Ireland, plus US and Australian accreditation [1], [3]. No Asian university rubric was found.
- **Shared sources.** Three agreements between the researchers rest on a source both found: MoSCoW [15], accessibility [13] and scope change [31]. They are not independently confirmed.
- **Partial reading.** Only abstracts were read for [16], [28] and [31]. The full text of [28] is embargoed until 12 November 2026.
- **Older sources.** [18] (2000), [10] (2010), [22] (2009) and the 1994 study behind [30] fall outside the brief's 2015 window.
- **Other gaps.** The OWASP 2025 edition was not checked [23]. [EVIDENCE GAP: cost and benefit of deployment as a student-project addition] [EVIDENCE GAP: effort data for sustainability, security review and documentation in student projects]

8 References

[1] ABET. (2025). Criteria for Accrediting Engineering Programs, 2025–2026. ABET Engineering Accreditation Commission. https://www.abet.org/accreditation/accreditation-criteria/criteria-for-accrediting-engineering-programs-2025-2026/ (accessed 14 October 2026).

[2] BCS, The Chartered Institute for IT. (2020). Guidelines on course accreditation: Information for universities and colleges. BCS, The Chartered Institute for IT. https://www.bcs.org/media/9006/course-accreditation-guidelines.pdf (accessed 14 October 2026).

[3] Engineers Australia. (2019). Stage 1 Competency Standard for Professional Engineer. Engineers Australia. https://WWW.ENGINEERSAUSTRALIA.ORG.AU/sites/default/files/2022-07/stage-1-competency-standard-professional-engineer.pdf (accessed 14 October 2026).

[4] Trinity College Dublin, Department of Computer Science. (n.d.). Final Year Project Evaluation Form and Marking Sheet (sample). Trinity College Dublin. https://projects.scss.tcd.ie/wp-content/uploads/sites/3/2020/09/FYP-Report-Marking-Sheet-Sample.pdf (accessed 14 October 2026).

[5] University of Oxford, Department of Computer Science. (2022). Examination Conventions, Parts A and B, 2022–23 (v1.1). University of Oxford. https://www.cs.ox.ac.uk/files/13739/Conventions%20Parts%20A%20and%20B%20202223%20-%20v1.1.pdf (accessed 14 October 2026).

[6] University of Warwick. (2026). CS310-30 Computer Science Project (module page). University of Warwick. https://courses.warwick.ac.uk/modules/2026/CS310-30?plain=true (accessed 14 October 2026).

[7] University College London. (2026). COMP0029 Individual Project for Year 3 BSc (module catalogue, 2026/27). University College London. https://www.ucl.ac.uk/module-catalogue/modules/individual-project-for-year-3-bsc-COMP0029 (accessed 14 October 2026).

[8] Aberystwyth University, Department of Computer Science. (2022). Major and Minor Project modules: 1. Introduction (CS39440/CS39620). Aberystwyth University (institution and year inferred from the URL). https://teaching.dcs.aber.ac.uk/docs/2022/MMP/information/introduction.html (accessed 14 October 2026).

[9] Sedelmaier, Y. and Landes, D. (2020). Analyzing Challenges in Software Engineering Capstone Projects. ICSEA 2020, IARIA. https://www.academia.edu/68314244/Analyzing_Challenges_in_Software_Engineering_Capstone_Projects (accessed 14 October 2026).

[10] Alaways, L. (2010). Factors that Inhibit or Enable Success of Capstone Design Teams. Fall 2010 Mid-Atlantic ASEE Conference. https://peer.asee.org/factors-that-inhibit-or-enable-success-of-capstone-design-teams.pdf (accessed 14 October 2026).

[11] Howe, E. (2018). Pitfalls to Avoid in Industry Capstone Project. 2018 Conference for Industry and Education Collaboration (ASEE CIEC). https://peer.asee.org/pitfalls-to-avoid-in-industry-capstone-project.pdf (accessed 14 October 2026).

[12] Hassani, H., Kadir, G., Al-Salihi, N. K., Monnet, W. M. A., Ali-Yahiya, T. and Alizade, F. (2018). Supervision of Undergraduate Final Year Projects in Computing: A Case Study. Education Sciences 8(4):210, DOI 10.3390/educsci8040210. https://www.mdpi.com/2227-7102/8/4/210 (accessed 14 October 2026).

[13] WebAIM. (2026). The WebAIM Million: The 2026 report on the accessibility of the top 1,000,000 home pages. WebAIM, Utah State University. https://webaim.org/projects/million/ (accessed 14 October 2026).

[14] Budiu, R. (2021). Why 5 Participants Are Okay in a Qualitative Study, but Not in a Quantitative One. Nielsen Norman Group. https://www.nngroup.com/articles/5-test-users-qual-quant/ (accessed 14 October 2026).

[15] Agile Business Consortium. (2026). What is MoSCoW Prioritization? (dated 28 May 2026). Agile Business Consortium. https://www.agilebusiness.org/?p=1907 (accessed 14 October 2026).

[16] Bijlsma, A., Doorn, N., Passier, H. J. M., Pootjes, H. J. and Stuurman, S. (2021). How do Students Test Software Units? 2021 IEEE/ACM ICSE-SEET, pp. 189–198, DOI

10.1109/ICSE-SEET52601.2021.00029.
https://research.ou.nl/en/publications/how-do-students-test-software-units-2/ (accessed 14 October 2026).

[17] Chisnall, D. (n.d.). Advice for project students. Department of Computer Science and Technology, University of Cambridge. https://www.cl.cam.ac.uk/~dc552/projects/advice.html (accessed 14 October 2026).

[18] Nielsen, J. (2000). Why You Only Need to Test with 5 Users. Nielsen Norman Group. https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/ (accessed 14 October 2026).

[19] University of Cambridge, Department of History and Philosophy of Science. (n.d.). Human participants (research ethics). University of Cambridge. https://www.hps.cam.ac.uk/about/admin/human-participants (accessed 14 October 2026).

[20] World Wide Web Consortium (W3C). (2024). Web Content Accessibility Guidelines (WCAG) 2.2. W3C Recommendation, 12 December 2024. https://w3.org/tr/wcag/ (accessed 14 October 2026).

[21] Matthies, C., Treffer, A. and Uflacker, M. (2017). Prof. CI: Employing Continuous Integration Services and GitHub Workflows to Teach Test-driven Development. 2017 IEEE Frontiers in Education (FIE), pp. 1–8 (arXiv copy). https://arxiv.org/pdf/1809.00580 (accessed 14 October 2026).

[22] Chang, J. (2009). Exploding Software-Engineering Myths. Microsoft Research. https://www.microsoft.com/en-us/research/blog/exploding-software-engineering-myths/ (accessed 14 October 2026).

[23] Open Worldwide Application Security Project (OWASP). (2021). OWASP Top 10:2021. OWASP. https://owasp.org/Top10/2021/ (accessed 14 October 2026).

[24] University of Sussex, School of Engineering and Informatics. (2022). Final Year Projects: Information for supervisors. University of Sussex. https://sussex.ac.uk/ei/internal/forstudents/informatics/undergraduate/finalyearprojects/informationforsupervisors (accessed 14 October 2026).

[25] University of Kent. (n.d.). Supervision: Introduction (EPCOS teaching resource bundle). University of Kent Computing. https://www.cs.kent.ac.uk/national/EPCOS/bundles/supervision-intro.html (accessed 14 October 2026).

[26] Green Software Foundation. (2024). Software Carbon Intensity (SCI) Specification, ISO/IEC 21031:2024. Green Software Foundation. https://greensoftware.foundation/standards/sci/ (accessed 14 October 2026).

[27] Howe, S., Rosenbauer, L., Ford, J. D., Alvarez, N., Paretti, M., Gewirtz, C., Kotys-Schwartz, D., Knight, D. and Hernandez, C. (c. 2018). Preliminary Results from a Study Investigating the Transition from Capstone Design to Industry. NSF Public Access Repository (venue not stated). https://par.nsf.gov//servlets/purl/10065279 (accessed 14 October 2026).

[28] Riddle, S., Forshaw, M., Zarb, M. et al. (2026, in press). The Computing Dissertation: Lessons from a UK Practitioner Network. Proceedings of the 2nd ACM Virtual Global Computing Education Conference. https://repository.falmouth.ac.uk/6529/ (accessed 14 October 2026).

[29] Reddington, J. (2025). Things I say to project students. Personal blog. https://joereddington.com/2025/10/30/things-i-say-to-project-students.html (accessed 14 October 2026).

[30] Vijayakumar, S. (2017). Things always take longer than expected. Always. Donders Wonders blog, Donders Institute, Radboud University. https://blog.donders.ru.nl/things-always-take-longer-than-expected-always/?lang=en (accessed 14 October 2026).

[31] Jaeger-Helton, K. and Smyser, B. M. (2017). Switching Midstream, Floundering Early, and Tolerance for Ambiguity: How Capstone Students Cope with Changing and Delayed Projects. 2017 ASEE Annual Conference & Exposition. https://peer.asee.org/switching-midstream-floundering-early-and-tolerance-for-ambiguity-how-capstone-students-cope-with-changing-and-delayed-projects (accessed 14 October 2026).

[32] Bona, F., Chanin, R., Nascimento, N. and Sales, A. (2023). Understanding the Gaps in Software Engineering Education from the Perspective of IT Leaders: A Field Study. CSEDU 2023, vol. 2, pp. 511–518. SciTePress. https://www.scitepress.org/Papers/2023/119591/119591.pdf (accessed 14 October 2026).

[33] Mountain Goat Software. (2016). Are 64% of Features Really Rarely or Never Used? Mountain Goat Software. https://www.mountaingoatsoftware.com/blog/are-64-of-features-really-rarely-or-never-used (accessed 14 October 2026).